Rules

ALDialog uses a list of written rules in order to manage the flow of the conversation between the human and the robot.

These rules are of two types: **User rules and Proposal rules**

- A User rule links a specific user input to possible robot output.

  u: (Hello Nao how are you today) Hello human, I am fine thank you and you?

- A Proposal rule triggers a specific robot output without any user output beforehand.

  proposal: Did you seen that guy on the TV yesterday?

  u1: (yes) He was crazy, no?

  u1: (no) Really, I need to tell you about him.

The User rule triggers a robot response by human input, the proposal is triggered usually by a sensor detecting a human presence and the robot speaks first (we will have the user initiated the conversation for the next assignment).

USER RULES

User rules have three parts, the level of the rule, the human input, and the robot output. The level of the rule will be discussed later in this doc. The human input or robot output can come from a list of alternatives. For example

u:(human input) robot output

User rule

Syntax

```
u:(Input) Answer

u:(Input)  Answer
```

Where:

- *Input* is the Human input,
- *Answer* is the Robot output.

Indentation and blank lines are ignored.

Usage

Makes the robot say or do the *Answer* when he hears the *Input*.

Effect

If the Human input matches, then the topic that contains the rule takes the focus.

For further details, see: Focus below.

Conditions

The topic containing the user rules must be activated to answer.

Example

```
topic: ~introduction ()

u:(hello) hello human

u:(how are you) I feel tired, my batteries are low
```

Execution

```
> hello
```

```
hello human

> how are you

I feel tired, my batteries are low
```

User subrule

Syntax

```
u:(input1) answer

   u1:(input2) answer

   u1:(input3) answer

     u2:(input4) answer

        u3:(input5) answer

     u2:(input6) answer



proposal: sentence

   u1:(input7) answer

   u1:(input8) answer
```

Where **u1:**, **u2:** and **u3:** are user subrules.

The indentation underlines the relationship between a rule and its subrules.

Usage

Allows creating several conversational contexts, making some rules activated if and only if a main User rule or Proposal has previously matched.

Activation / deactivation

At first, the User rules and Proposal are activated while **subrules** are deactivated.

- If a User rule matches and if it has a scope, its **subrules** (**u1:**) are activated; the other User rule remain activated.
- If one of the **u1:** subrule matches, then its own subrules (**u2:**) are activated, and the first-level rules (**u1:**) are deactivated.
- At anytime, if another User rule matches, then the previous subrules are deactivated.

Scope

The subrules of a rule are called the **scope** of the rule.

```
u:(input1) answer                    Scope of the rule u:
  u1:(input2) answer
  u1:(input3) answer      Scope of the rule u1:
    u2:(input4) answer
       u3:(input5) answer
    u2:(input6) answer

proposal: sentence
   u1:(input7) answer
   u1:(input8) answer
```

Example

```
topic: ~introduction ()

u:(talk about animals) do you have a cat or a dog?

   u1:(dog) is it a big dog?

     u2:(yes) make sure he has enough space to run

     u2:(no) it is so cute

   u1:(cat) do you live in the countryside?

     u2:(yes) does your cat goes outside?

        u3:(yes) does he hunt mouses?

     u2:(no) i hope your flat is big enough

   u1:(none) neither do I


u:(talk about sport) what a good idea


proposal: Do you want to talk about sport?

   u1:(yes) Cool
```

```
    u1:(no) OK
```

Execution

```
> talk about animals

do you have a cat or a dog?

> I have a cat

do you live in the countryside?

> no

I hope your flat is big enough

> Now I want to talk about sport

what a good idea
```

Proposal

Syntax

```
proposal: sentence
```

Where:

- *sentence* is a Robot output.

Usage

While rules are triggered by a Human input, to be said, a **Proposal** needs to be triggered with one of the **topic progression** functions:

- ^nextProposal,
- ^previousProposal,
- ^sameProposal,
- ^goto,
- ^gotoRandom,
- ^gotoReactivate, or
- ^topicTag.

Activation/deactivation

Conditions

The Topic containing the user rules must be active.

Example

```
topic: ~introduction ()

proposal: take a cup and fill it with milk

proposal: add 3 strawberries

proposal: add some vanilla ice cream

proposal: close the cup with the lid and shake it

proposal: you did it! you can enjoy your milkshake



u:(I want a milkshake)

    ok, let's do it.

    follow my instruction and say next when you are ready for
the next step.



u:(next) ^nextProposal
```

Execution

```
> I want a milkshake

ok, let's do it.

follow my instruction and say next when you are ready for the
next step.

>next

take a cup and fill it with milk

>next

add 3 strawberries

>next
```

```
add some vanilla ice cream

>next

close the cup with the lid and shake it

>next

you did it! you can enjoy your milkshake

>next

(no answer)
```

Delimiters

Choice: [ ]

Syntax

```
u:([word1 word2 wordn]) answer

u:(input) [word1 word2 wordn]

u:([word1 word2 wordn]) [word1 word2 wordn] human
```

Where:

- *word1* to *wordn* are words that could substitute with each other.

Usage

- In a Human input: creates one rule accepting variations.
- In a Robot output: if the rule is triggered several times, the words will be used sequentially in order to create varying responses.

Example

```
topic: ~introduction ()

u:([hi hello]) [hello hi] human
```

The Human input can be 'hello' or 'hi'. The Robot output can be 'hello human' or 'hi human'. By default answers are in a sequential order.

Execution

```
> hello
```

```
hello human

> hello

hi human

> hi

hello human
```

Phrase delimiter: ” “

Syntax

```
u:([word1 "phrase 1"]) ["phrase 2" "phrase 3"]
```

Where:

- *word1* is a word that could substitute with *phrase 1*.
- *phrase 2* and *phrase 3* are phrases used sequentially in order to create varying responses.

Usage

Allows you to place a phrase instead of a single word in a choice [] or an optional part {}.

Example

```
topic: ~introduction ()

language:enu



u:(["hello how are you" "hello are you OK"]) ["I am fine" "I am
OK"]
```

Execution

```
> hello are you OK

I am fine

> hello how are you

I am OK
```

Optional part: { }

Syntax

```
u:(beginning of the sentence {optionalWord} end of the sentence) answer

u:(beginning of the sentence {"optional phrase"} end of the sentence) answer
```

Where:

- *optionalWord* is a word that could appear or not in the Human input.
- *optional phrase* is a phrase that could appear or not in the Human input.

Usage

Optional word or phrase, at the beginning, the middle or the end of a sentence.

Conditions

Inside a Choice: [ ], optional word can only be used inside a phrase:

- **Not correct**: [chocolate milk {bread}]
- **Correct**: [chocolate milk "{white} bread"]

Example

```
topic: ~introduction ()

language:enu



u:(hello {buddy} how are you) hello I am fine
```

Execution

```
>hello how are you

hello I am fine

>hello buddy how are you

hello I am fine
```

Special characters

Comments:

Syntax

```
# comment
```

Comment is defined by character #. All the characters following # in the same line will be ignored.

Wildcard: *

Syntax

```
u:(sentence * sentence) answer
```

Usage

Matches any word or phrase.

Use wildcards sparingly

Too many wildcards can degrade performance of the automatic speech recognition.

Example

```
topic: ~introduction ()

u:(my name is *) nice to meet you

u:(I like to * a lot) it sounds cool
```

Execution

```
> my name is David

nice to meet you

> I like to dance a lot

it sounds cool

> I like to play tennis a lot

it sounds cool
```

Forbidden word: !

Syntax

```
u:(sentence !forbiddenWord) answer
```

Usage

Allows you to define a forbidden word in a Human input.

Example

```
topic: ~introduction ()



u:(tell me a story) OK, a new one?

u:(!don't tell me a new one) great, I'll tell you a new story!

u:(don't tell me a new one) Ok, so I'll tell you the story of
Peter Pan.
```

Execution

```
> tell me a story

OK, a new one?

> tell me a new one

great, I'll tell you a new story!
```

Input storing: _

Syntax

```
u:(sentence _[word1 word2]) answer $1


u:(sentence _~concept) answer $1


u:(sentence _*) answer $1
```

Where:

- *_[word1 word2]* is the choice of words the robot will be able to catch and reuse.
- *$1* is the first caught word.

  If there are several _, *$2* is the second caught word, *$3* is the third, etc. There is no limitation in variable number.
- *_~conceptName* allows catching any word included in a concept. For further details, see: Concept call: ~.
- _* allows catching any word said using the speech recognition.

Warning

_* is not available for all robot yet, it requires a specific speech recognition license.

This is from the NAO robots, we don't have the same license problems.

Note: giving a choice of words to recognize eases the speech recognition.

Usage

Allows catching one or several words in the Human input and reuse it within the Robot output directly following.

Tips: if you need to reuse the value elsewhere in the dialog, store it in a variable.

```
u:(my name is _*) nice to meet you $1 $name=$1
```

Example

```
topic: ~introduction ()

language:enu
```

```
u:(my name is _*) nice to meet you $1

u:(I like _[chocolate cheese]) do you want to eat $1 now?
```

Execution

```
> my name is David

nice to meet you David

> I like cheese

do you want to eat cheese now?

> I like chocolate

do you want to eat chocolate now?
```

Variable: $

Syntax

```
# Affect a variable

u:(input) answer $variableName=value



# Use a variable in a robot answer

u:(input) answer $variableName
```

Where:

variableName is the name of the variable.

Usage

Variable can be used the same way in Human input or Robot output.

Variables are global, i.e. they can be used in several behaviors.

They don’t have a type and don’t need to be declared, they can be directly affected and used in rules.

They can be:

- set
- used in a Robot output
- used in conditions (see: Conditions: == > <> <).

Example

```
topic: ~introduction ()



   u:(I want some _[chocolate cheese]) OK $askedFood=$1

   u:(what did I ask) ^first["you asked $askedFood" "I don't
know"]
```

Execution

```
> what did I ask

I don't know

> I want some chocolate

OK

> what did I ask

you asked chocolate
```

Conditions: == > <> <

Syntax

```
# condition on variable

u:($variableName==value) answer



# answer if variable>value

u:(input) answer $variableName>value
```

```
# answer if variable<value

u:(input) answer $variableName<value


# answer if variable different from value

u:(input) answer $variableName<>value


# condition on variable in human input

u:("input $variableName==value") answer


# condition on variable in answer

u:(input) ["answer $variableName==value" "another answer"]
```

Usage

Placed in a Human input or in a Robot output, a condition:

- triggers the rule if true,
- blocks the rule if false.

A condition based on an empty variable will be ignored.

The condition ordering does not matter.

If condition is false anywhere in quote, then the sentence in quote will not be said.

Example

```
topic: ~introduction ()

   u:(I want some _[chocolate cheese]) OK, you want some $1
$askedFood=$1

   u:(what did I ask) ^first ["you asked $askedFood" "I don't
know"]

   u:(can I have more)
```

```
   ^first["$askedFood==chocolate sorry, too much chocolate could
hurt you"

   "yes, please take more $askedFood"]
```

See also: ^first.

Execution

```
> I want some chocolate

OK, you want some chocolate

> can I have more

sorry, too much chocolate could hurt you

> I want some cheese

OK, you want some cheese

> can I have more

yes, please take more cheese
```

Robot events: e:eventName

Syntax

```
u:(e:event) answer

# will answer whenever the event "event" is raised



u:([e:event sentence]) answer

# will answer whenever the event "event" is raised or the
sentence is said.



u:("e:event sentence") answer

# will answer whenever the event "event" is raised and then the
sentence is said
```

```
# (the event must be raised before saying the sentence)
```

Note: For each event, `e:eventName` a variable `$eventName` is also available.

Note: in Choregraphe qiChat events and input/output can have the same name. If an event have the same name than box input/output, the variable is only linked to the box and is not anymore usable with **ALMemory**.

Usage

Catches an event.

Allows to catch:

- any NAOqi Events providing information related to the robot: movement, battery information, face information,
- more specifically any ALDialog event list providing information about the Dialog engine,
- but also the events raised each time a value is assigned to a variable.

Note

Robot output cannot occur at high frequency, both data and event are merged.

Conditions

The subscribe will only occur when the topic is activated. The unsubscribe will occur when the topic is deactivated.

Example

```
topic: ~introduction ()

language:enu



u:(be my puppet) ok, touch my leg or ask me

u:(e:LeftBumperPressed "move this leg") ok I move the left leg

u:(e:LeftBumperPressed "light this leg") ok I turn leds on left
leg
```

Execution

```
>be my puppet

ok, touch my leg or ask me

>(user touch the left bumper)
```

```
ok I move the left leg

>(user touch the left bumper) light this leg

ok I turn leds on left leg
```

Concept call: ~

Syntax

```
u:(~conceptName) answer

u:(input) answer ~conceptName
```

Usage

A **Concept** can be used both in Human input or Robot output to replace a list of item previously declared.

- In Human input, the rule will match if anyone of the items of the list is recognized.
- In Robot output, one item of the list will be said. Each call of the rule will call sequentially another item of the list.

Conditions

the **Concept** must be previously defined.

Example

```
topic: ~introduction ()



   concept:(want) [want need desire like]

   concept:(alcohol) [beer vodka "red wine"]

   concept:(child-drink) ["orange juice" milk]

   concept:(drink) [~alcohol ~child-drink]



   u: (what do you do) I can help people with drinks



   u:(I ~want to drink) what you ~want to drink?
```

```
u1:(* _~drink) do you want me to search $1 for you?
```

Execution

```
> what do you do

I can help people with drinks

> I need to drink

what do you want to drink?

> I want red wine

do you want me to search red wine for you?
```

^clear

Syntax

```
u:(input) ^clear(variableName)
```

Usage

Clears a variable. A cleared variables cannot be said. A sentence with cleared variable cannot be said. A condition with cleared variable is always false.

Rule functions and properties

^activate ^deactivate

Syntax

```
u:(input) answer ^deactivate(tag)

u:(input) answer ^activate(tag)
```

Usage

^deactivate function deactivates a tag, **^activate** function reactivates a tag.

If a Robot output contains a deactivated tag, this Robot output cannot be triggered.

Example

```
topic: ~introduction()
```

```
language: enu


proposal: %greeting1 hello

proposal: nice to meet you

u:(hello) hello ^deactivate(greeting1)

u:(say something) ^nextProposal
```

Execution

```
> Hello

Hello

> say something

nice to meet you
```

Example

```
topic: ~introduction ()

language:enu


u:(my name is _*) nice to meet you $name=$1

u:(what is my name) ^first["your name is $name" "I don't know"]

   u1:(no) OK ^clear(name)
```

Execution

```
> my name is David

nice to meet you

> what is my name
```

```
your name is David

> no

OK

> what is my name

I don't know
```

Related functions

- ^first.

^exist ^notExist

Syntax

```
topic: ~exist()

language: enu

u:(hello) ["^exist(42) the variable exist" "^notExist(42) the variable doesn't exist $42=1"]
```

Usage

A sentence that contains ^exist won't be said if the variable parameter doesn't exist or is an empty string.

^first

Syntax

```
u:(input) ^first [answer1 answer2]



# Inside a choice, requires quotes

u:(input)  [answer1 "^first [answer2 answer3]"]
```

Usage

When added before a Choice: [ ] in the Robot output part, instead of using each answer in turn, the interpreter always use the first valid Robot output.

A valid Robot output is an answer:

- without any condition nor non-affected variable,
- including affected variable(s),
- including a condition tested as true.

Using a non-affected variable in a robot output will make the rule invalid, i.e. the rule will not be said at all.

To avoid this situation, use the function ^first before a choice [ ].

This function tries to say the first valid output. If the first element of the choice is invalid because it contains a non-affected variable, it will try the second element, and so on.

Example

```
topic: ~introduction ()

language:enu



u:(my name is David) nice to meet you David $name=David

u:(my name is Paul) nice to meet you Paul $name=Paul

u:(what is my name) ^first["your name is $name" "I don't know"]
```

Execution

```
> what is my name

I don't know

> my name is David

nice to meet you David

> what is my name

your name is David
```

.

^goto

Syntax

```
u:(input1) %tag Answer
```

```
u:(input2) ^goto(tag)
```

Usage

The **^goto** function allows you to:

- jump to a Robot output (in a user rule or an activated proposal) containing a specific tag.
- It can also be used to declare two Human inputs as equivalent.
- To jump to a deactivated Proposal, use ^gotoReactivate.

Example

```
topic: ~introduction ()



u:(hello how are you) hello ^goto(how)

u:(how you are) %how I'm fine
```

Execution

```
> how are you

I'm fine

> hello how are you

hello I'm fine
```

^gotoRandom

Syntax

```
u:(input) ^gotoRandom(tag)

proposal: %tag answer1

proposal: %tag answer2
```

Usage

The **^gotoRandom** function is identical to ^goto except if the tag is used in several rules, it will chose one randomly.

Example

```
topic: ~introduction ()

u:(hello) ^goto(hi)

proposal %hi hello

proposal: %hi welcome

proposal: %hi hey
```

Execution

```
> hello

hey

> hello

hello

> hello

welcome
```

gotoReactivate

Syntax

```
proposal: %tag Answer

u:(input) ^gotoReactivate(tag)
```

Usage

The **^gotoReactivate** function allows you to jump to a deactivated Proposal and reactivate it.

Example

```
topic: ~introduction ()

language:enu



proposal: %love I love you
```

```
u:(do you love me) yes, ^gotoReactivate(love)

u:(are you sure) sure, ^gotoReactivate(love)
```

Execution

```
I love you

> do you love me

yes, I love you

> are you sure

sure, I love you
```

^nextProposal

Syntax

```
u:(input) answer ^nextProposal
```

Usage

The function **^nextProposal** says the first activated proposal in the Topic which has the Focus.

Example

```
topic: ~introduction ()

language: enu



u:(hi) hello ^nextProposal



proposal: how are you?

  u1:(I'm fine) cool!

  u1:(I'm tired) oh, you should take a nap
```

```
proposal: again!
```

Execution

```
> hi

hello how are you?

> I'm tired

oh, you should take a nap

> hi

hello again!
```

^play

Syntax

```
topic: ~introduction ()

language: enu

u:(play * sound) ^play(sound)
```

Usage

^play is a beta function that can be removed at any version. ^play allows playing a sound with audio device soundSet instead of animated speech.

^previousProposal

Syntax

```
u:(input) answer ^previousProposal
```

Usage

The function **^previousProposal** repeats the Proposal said previously to the last in the same Topic.

Example

```
topic: ~introduction ()

language: enu
```

```
u:(hi) hello ^nextProposal


proposal: how are you?
  u1:(I'm fine) cool!
  u1:(I'm tired) oh ^nextProposal


proposal: do you want some tea?
   u1:(yes) I'm bringing you a cup of tea.
   u1:(no) okay, ^nextProposal


proposal: what about some coffee?
   u1:(yes) I'm bringing you a cup of coffee.
   u1:(no) okay, maybe you should just take a nap.
   u1:(can you repeat please) ^sameProposal ^stayInScope
   u1:(what did you say before) ^previousProposal ^stayInScope
```

Execution

```
> hi
hello how are you?
> I'm tired
do you want some tea?
> no
OK, what about coffee?
```

```
> can you repeat please

OK, what about coffee?

> what did you say before

do you want some tea?
```

^private

Syntax

```
u:^private(Input) Answer
```

Effect

Apply a private status to a rule. A private **user rule** is only active when its corresponding topic has the Focus.

Example

```
topic: ~music ()

language:enu



u:(I want to talk about music) OK!

u:^private(what are we talking about) we are talking about music
```

Execution

```
> I want to talk about music

OK!

> what are we talking about

we are talking about music
```

^rand -

Syntax

```
u:(input) ^rand[answer1 answer2]
```

Usage

Can be added before a choice [] in the Robot output.

Instead of using each answer sequentially, the interpreter will choose randomly a valid Robot output.

A valid Robot output is:

- an answer including a condition tested as true or
- an answer including no condition at all.

Rand function in a choice needs to be inside quote.

For further details, see: Conditions: == > <> <.

Example

```
u:(hello) ^rand[hello hi "what's up" "hey there"]
```

^repeat

Syntax

```
u:(^repeat[word1 word2 word3]) answer
```

Usage

Allow a free combination of listed words.

The Human input is recognized if one or several word(s) among the list are said, regardless of the order.

^sameProposal

Syntax

```
u:(input) answer ^sameProposal
```

Usage

The function **^sameProposal** repeats the last proposal said in the same topic.

Best practices

When using **^sameProposal**, always make the robot say something like 'Ok I'll repeat', 'No problem,' so that the user knows what to expect. It also makes the repetition clearer.

Example

```
topic: ~introduction ()
```

```
language: enu


u:(hi) hello ^nextProposal


proposal: how are you?

  u1:(I'm fine) cool!

  u1:(I'm tired) oh ^nextProposal


proposal: do you want some coffee?

   u1:(yes) I'm bringing you a cup of coffee.

   u1:(no) okay, maybe you should just take a nap.

   u1:(can you repeat please) Ok I'll repeat. ^sameProposal
```

Execution

```
> hi

hello how are you?

> I'm tired

do you want some coffee?

> can you repeat please

Ok I'll repeat. Do you want some coffee?

> yes

I'm bringing you a cup of coffee.
```

^stayInScope

Syntax

```
u:(input) answer

   u1:(input1) answer ^stayInScope

   u1:(input2) answer
```

Usage

Used in a rule or subrule to stay in the current scope when the corresponding rule has matched.

Example

```
u:(let's talk about animals) do you have a cat or a dog?

   u1:(I {also} have a dog) dogs are funny ^stayInScope

   u1:(I {also} have a cat) I once met a cat  ^stayInScope
```

Execution

```
> let's talk about animals

do you have a cat or a dog?

> I have a dog

I love dogs

> I also have a cat

I once met a cat
```

Animated speech functions

^switchFocus

Stops dialog engine and runs a **Behavior** identified by application **uuid** and **Behavior** name.

Warning

If behavior is at the root of the application, then switch focus parameter is ^switchFocus(uuid/.)

Warning

Life needs to be started.

Syntax

```
topic: ~addition()

language: enu



# we suppose an application weather is installed on robot.

u:(start weather)
^switchFocus(weatherApplication/weatherBehavior)
```

^mode

Changes the body language mode for the current text.

Syntax

```
u:(input) answer1 "^mode(body_language_mode) word1 word2..."
answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

^run

Suspends the speech, runs an animation and resumes the speech.

Syntax

```
u:(input) answer1 ^run(animation_full_name) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

Related functions

- ^start,
- ^stop,
- ^wait.

^runTag

Suspends the speech, runs an animation which has the mentioned tag and resumes the speech.

Syntax

```
u:(input) answer1 ^runTag(tag_name) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

Related functions

- ^startTag,
- ^stopTag,
- ^waitTag.

^runSound

Suspends the speech, plays a sound and resumes the speech.

Syntax

```
u:(input) answer1 ^runSound(soundSet/soundFile) answer2

u:(input) answer1 ^runSound(soundSet/soundFile, soundVolume)
answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

Related functions

- ^startSound,
- ^stopSound,
- ^waitSound.

^start

Starts behaviors or animations.

Syntax

```
u:(input) answer1 ^start(behavior) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

Related functions

- ^run,
- ^stop,
- ^wait.

^startTag

Starts an animation which has the mentioned tag.

Syntax

```
u:(input) answer1 ^startTag(tag_name) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

Related functions

- ^runTag,
- ^stopTag,
- ^waitTag.

^startSound

Plays a sound in background.

Syntax

```
u:(input) answer1 ^startSound(soundSet/soundFile) answer2

u:(input) answer1 ^startSound(^startSound(soundSet/soundFile,
soundVolume)) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

Related functions

- ^runSound,
- ^stopSound,
- ^waitSound.

^stop

Stops a running behavior or an animation.

Syntax

```
u:(input) answer1 ^stop(behavior) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

Related functions

- ^run,
- ^start,
- ^wait.

^stopTag

Stops a running animation which has the mentioned tag.

Syntax

```
u:(input) answer1 ^stopTag(tag_name) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

Related functions

- ^runTag,
- ^startTag,
- ^waitTag.

^stopSound

Stops a sound.

Syntax

```
u:(input) answer1 ^stopSound(soundSet/soundFile) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

Related functions

- ^runSound,
- ^startSound,
- ^waitSound.

^wait

Waits until a running behavior or an animation has finished.

Syntax

```
u:(input) answer1 ^wait(animation) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

- ^run,
- ^start,
- ^stop.

^waitTag

Suspends the speech, waits for the end of the animation which has the mentioned tag and resumes the speech.

Syntax

```
u:(input) answer1 ^waitTag(tag_name) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

- ^runTag,
- ^startTag,
- ^stopTag.

^waitSound

Suspends the speech, waits for the end of the sound and resumes the speech.

Syntax

```
u:(input) answer1 ^waitSound(soundSet/soundFile) answer2
```

For further details, see ALAnimatedSpeech: Annotated Text.

- ^runSound,
- ^startSound,
- ^stopSound.

Topic properties

^notInEmbeddedASR

Syntax

```
topic: ~introduction ^notInEmbeddedASR ()
```

Usage

Prevents the automatic loading of the rules in the speech recognition engine.

By default, the **user rules** contained in a topic are loaded into the embedded speech recognition engine. If **^notInEmbeddedASR** property is used, none of the **user rules** are loaded into the embedded ASR engine. Very short words or rules with many * can trigger whatever we say.

^noPick

Syntax

```
topic: ~introduction ^noPick ()
```

Usage

Prevents the topic from getting randomly the Focus.

The **Topic** can still get the focus when a rule matches, but the **Dialog engine** cannot decide by itself to give the focus.

^noStay

Syntax

```
topic: ~introduction ^noStay ()plus
```

Usage

Prevents the topic from keeping the Focus.

If a rule matches in the topic, the Robot output is executed, but the Focus stays on the previous topic.

^resetOnFocus

Syntax

```
topic: ~introduction ^resetOnFocus ()
```

Usage

By default a proposal can be said only once.

^resetOnFocus reactivates all proposals of a **Topic** when:

- the topic gets the Focus and
- all his proposals has been previously deactivated.

^fallback

Syntax

```
topic: ~introduction ^fallback ()
```

Usage

Give the lowest priority to the rules contained in this topic.

Its rules will be taken in consideration after the rules of:

1. the main section of the Topic having the focus,
2. the main section of all Topics - ^fallback excepted,
3. the recover sections of all topics.

^topicTag

Syntax

```
^topicTag(topic,tag)
```

Usage

Allows to go in a specific location in a topic, defined by a tag.

Example

```
topic: ~topic1()

language: enu



proposal: %test test

topic: ~topic2()

language: enu



u:(test) ^topicTag(topic1,test)
```

Related functions

- ^activate ^deactivate,
- ^goto,
- ^gotoRandom,
- ^gotoReactivate,
- Tag: %.

Transformation rules

Skin: s:

Syntax

```
s:(Answer) Modifications

s:(Answer)

Modifications
```

Where:

- *Answer* is the Robot output we want to match to apply this skin

- *Modifications* the modifications ^addword or/and ^replace we want to apply to the answer

Usage

When a Robot output matches a skin the robot says the *Answer* after *Modifications* are applied.

^addword

Syntax

```
s:(Answer) ^addword(Expression, Position, Frequency)
```

Where:

- *Answer* is the Robot output to be matched
- *Expression* can be any string (word(s), concept...)
- *Position* is the position where you want to add the zexpression in the sentence. This parameter can be:
- an integer: for instance "2" if you want to put the *Expression* in the second position
- "end" if you want the *Expression* to be added at the end of the sentence
- "start" if you want the *Expression* to be added at the beginning of the sentence
- *Frequency* represents the probability of the skin to be applied (1:always apply, 0:never apply)

Usage

Add an *Expression* in each Robot output matching the skin.

Conditions

The rule must be active.

Example

```
topic: ~introduction ()

language:enu



u:(hello) hello human

u:(how are you) I am fine



s:(hello human) ^addword(I am your father, end, 1)

s:(*) ^addword(hey, start, 1)
```

Execution

```
> hello

hey hello human I am your father

> how are you

hey I am fine and you
```

^replace

Syntax

```
s:(Answer) ^replace(Expression1, Expression2, Frequency)
```

Where:

- *Answer* is the Robot output to be matched
- *Expression1* can be any string (word(s), concept...) you want to replace
- *Expression2* can be any string (word(s), concept...) you want to put instead of *Expression1*
- *Frequency* is the probability of the skin to be applied (1:always, 0:never)

Usage

Replaces *Expression1* with *Expression2* in each Robot output matching the skin.

Conditions

The rule must be active.

Example

```
topic: ~introduction ()

language:enu



u:(hello) hello human

u:(what do you like to do) I like to speak with you human

u:(do you want to play) yes human we can play together



s:({*} human {*}) ^replace(human, Michael, 1)
```

```
s:({*} speak {*}) ^replace(speak, play, 1)
```

If you are not familiar with {*} check: Wildcard: * to under Optional part: { }.

Execution

```
> hello human

hello Michael

> what do you like to do

I like to play with you Michael

> do you want to play

yes Michael we can play together
```